

# Messaging API

## ▼ Relevant source files

Backend/STUDER/src/main/java/facu/studer/DTOs/messages/ChatListPageResponseDTO.java

Backend/STUDER/src/main/java/facu/studer/DTOs/messages/ChatPreviewDTO.java

Backend/STUDER/src/main/java/facu/studer/DTOs/messages/DirectMessagePageResponseDTO.java

Backend/STUDER/src/main/java/facu/studer/DTOs/messages/DirectMessageRequestDTO.java

Backend/STUDER/src/main/java/facu/studer/DTOs/messages/DirectMessageResponseDTO.java

Backend/STUDER/src/main/java/facu/studer/DTOs/user/FriendResponseDTO.java

Backend/STUDER/src/main/java/facu/studer/DTOs/user/FriendsListResponseDTO.java

Backend/STUDER/src/main/java/facu/studer/controllers/DirectMessageController.java

Backend/STUDER/src/main/java/facu/studer/entities/Friend.java

Backend/STUDER/src/main/java/facu/studer/entities/messages/DirectMessage.java

Backend/STUDER/src/main/java/facu/studer/mappers/DirectMessageMapper.java

Backend/STUDER/src/main/java/facu/studer/services/FriendService.java

Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java

Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java

The Messaging API facilitates private communication between users. It implements a **Friend-gated messaging model**, where full conversational history and chat listings are restricted to confirmed friends, while allowing a "Message Request" flow for initial contacts from non-friends.

## Overview and Data Model

The messaging system is built around the **DirectMessage** entity, which tracks the sender, receiver, content, and read status. Relationships are strictly managed by the **Friend** entity state to ensure privacy.

## DirectMessage Entity

The `DirectMessage` entity inherits from `BaseEntity` and contains the following core fields:

- `sender` : Reference to the `User` who sent the message.
- `receiver` : Reference to the `User` receiving the message.
- `content` : The text body of the message.
- `sentAt` : Timestamp of the message.
- `isRead` : Boolean flag for read receipts.
- `replyTo` : Optional self-reference to another `DirectMessage` for threaded replies.

## Friend-Gated Logic

Messaging permissions are determined by the `Friend` relationship flags: `senderAccept` and

`receiverAccept` `Backend/STUDER/src/main/java/facu/studer/entities/Friend.java` 31-41

- **Confirmed Friends:** Both flags are `true`. Users can view full chat history and appear in each other's chat lists

`Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java` 80-82

- **Message Requests:** If a user sends a message to someone who hasn't accepted their friend request, it is treated as a "Message Request"

`Backend/STUDER/src/main/java/facu/studer/controllers/DirectMessageController.java` 35

### Sources:

- `Backend/STUDER/src/main/java/facu/studer/entities/messages/DirectMessage.java`
- `Backend/STUDER/src/main/java/facu/studer/entities/Friend.java` 22-42
- `Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java` 69-82

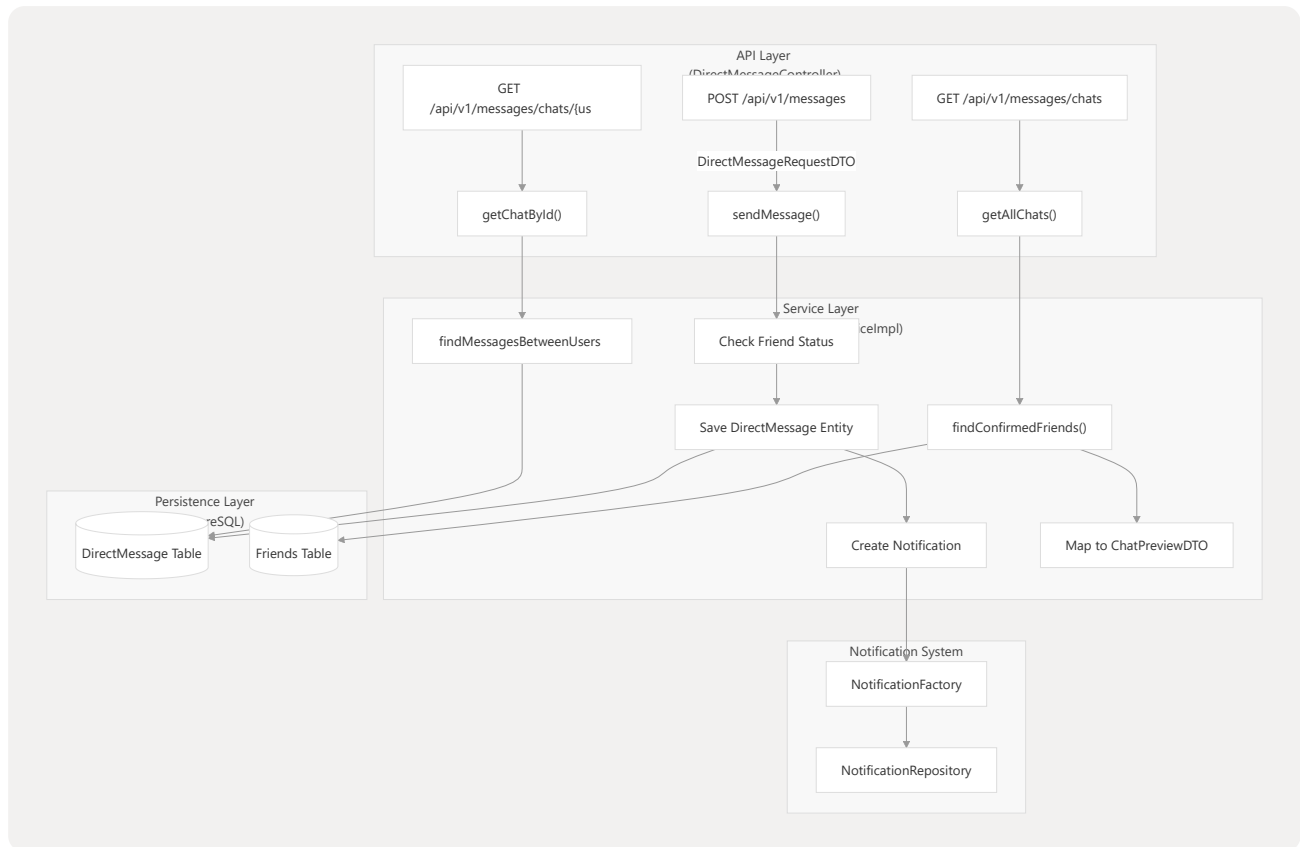
---

## System Architecture and Data Flow

The following diagram illustrates how the `DirectMessageController` interacts with the service layer and the underlying persistence entities.

## Messaging Service Flow

"This diagram maps the API Request space to the Service Implementation and Entity storage."



### Sources:

- Backend/STUDER/src/main/java/facu/studer/controllers/DirectMessageController.java 21-109
- Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java 39-153
- Backend/STUDER/src/main/java/facu/studer/repositories/DirectMessageRepository.java

## Core Service Logic

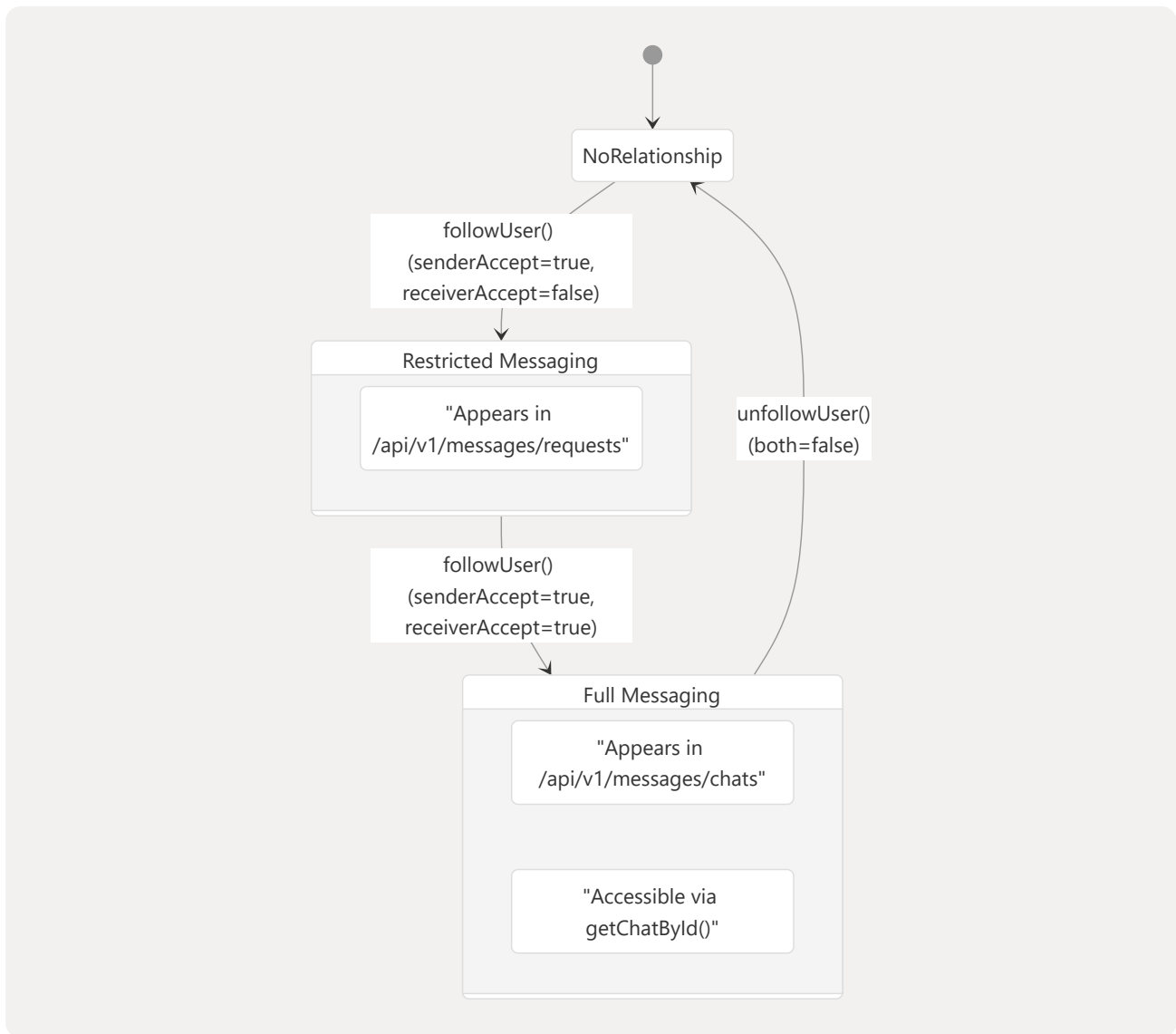
### Chat Management ( `DirectMessageServiceImpl` )

The service implementation handles the complexity of aggregating chat metadata (last message, unread count) for the conversation list.

| Method      | Logic Description                                                                                                                                                                                                                                                                               | Key DTO                      |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| getAllChats | Retrieves all <code>Friend</code> records where both users accepted. For each friend, it fetches the <code>findLastMessageBetweenUsers</code> and calculates <code>unreadCount</code><br>Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java 107-149 | ChatPreviewDTO               |
| getChatById | Validates that the <code>friendship</code> exists and is confirmed before returning a paginated list of <code>DirectMessageResponseDTO</code><br>Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java 69-98                                           | DirectMessagePageResponseDTO |
| sendMessage | Persists the message and triggers a <code>UserNotification</code> via <code>NotificationFactory</code> . If not friends, it creates a "Message Request" type<br>Backend/STUDER/src/main/java/facu/studer/controllers/DirectMessageController.java 33-36                                         | DirectMessageResponseDTO     |

## Friend Status and Messaging

"This diagram maps the natural language 'Friendship State' to the specific code flags in the Friend entity."



## Sources:

- Backend/STUDER/src/main/java/facu/studer/services/implementation/FriendServiceImpl.java 63-116
- Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java 79-82
- Backend/STUDER/src/main/java/facu/studer/entities/Friend.java 22-42

## API Endpoints

### 1. Send Message

POST /api/v1/messages

- **Request:** `DirectMessageRequestDTO` containing `receiverId`, `content`, and optional `replyToId`  
Backend/STUDER/src/main/java/facu/studer/DTOs/messages/DirectMessageRequestDTO.java 21-46
- **Validation:** Content must not be blank  
Backend/STUDER/src/main/java/facu/studer/DTOs/messages/DirectMessageRequestDTO.java 33

## 2. Get Chat List

GET /api/v1/messages/chats?page=0

- **Returns:** `ChatListPageResponseDTO`  
Backend/STUDER/src/main/java/facu/studer/DTOs/messages/ChatListPageResponseDTO.java 20-41
- **Data:** A list of `ChatPreviewDTO` objects. Each preview includes the other user's profile info, the `lastMessageContent`, `lastMessageTime`, and the `unreadCount`  
Backend/STUDER/src/main/java/facu/studer/DTOs/messages/ChatPreviewDTO.java 20-71

## 3. Get Conversation History

GET /api/v1/messages/chats/{userId}?page=0

- **Pagination:** Returns 30 messages per page  
Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java 41
- **Response:** `DirectMessagePageResponseDTO` containing a list of `DirectMessageResponseDTO`  
Backend/STUDER/src/main/java/facu/studer/DTOs/messages/DirectMessagePageResponseDTO.java 20-41

## 4. Mark as Read

PATCH /api/v1/messages/mark-as-read/{senderId}

- Updates all messages from the specified sender to `isRead = true` for the current user  
Backend/STUDER/src/main/java/facu/studer/controllers/DirectMessageController.java 103-108

### Sources:

- Backend/STUDER/src/main/java/facu/studer/controllers/DirectMessageController.java 40-109
- Backend/STUDER/src/main/java/facu/studer/DTOs/messages/ChatPreviewDTO.java 20-71

- Backend/STUDER/src/main/java/facu/studer/services/implementation/DirectMessageServiceImpl.java

41-42